

Take-Home R Practical Examination

Bavindu Gunasinghe

2024-11-14

R Markdown

This is an R Markdown document. Markdown is a simple formatting syntax for authoring HTML, PDF, and MS Word documents. For more details on using R Markdown see <http://rmarkdown.rstudio.com>.

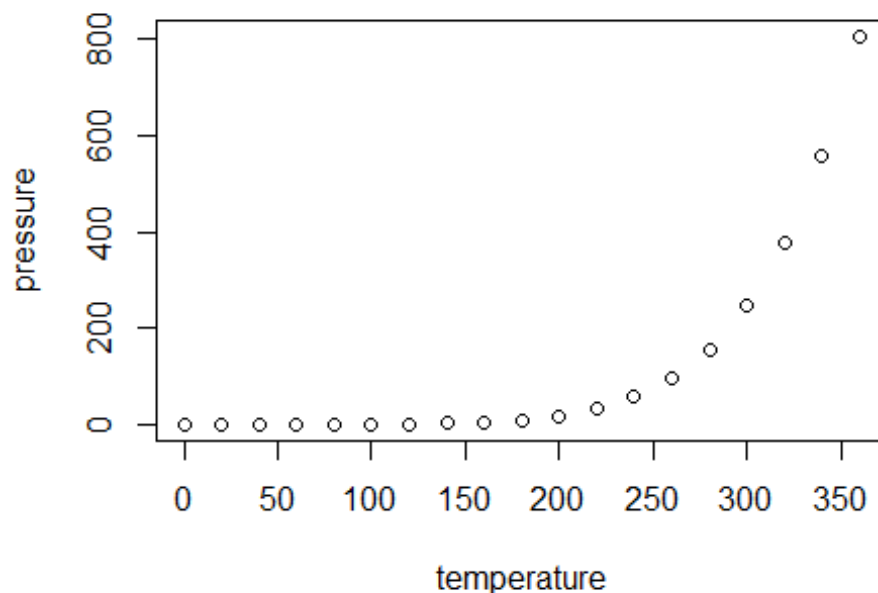
When you click the **Knit** button a document will be generated that includes both content as well as the output of any embedded R code chunks within the document. You can embed an R code chunk like this:

```
summary(cars)
```

```
##      speed      dist
##  Min.   : 4.0    Min.   :  2.00
##  1st Qu.:12.0    1st Qu.: 26.00
##  Median :15.0    Median : 36.00
##  Mean   :15.4    Mean    : 42.98
##  3rd Qu.:19.0    3rd Qu.: 56.00
##  Max.   :25.0    Max.    :120.00
```

Including Plots

You can also embed plots, for example:



Note that the `echo = FALSE` parameter was added to the code chunk to prevent printing of the R code that generated the plot.

```
# Load necessary libraries
```

```
library(tidyverse)
```

```
## Warning: package 'tidyverse' was built under R version 4.3.3
```

```
## Warning: package 'ggplot2' was built under R version 4.3.3
```

```
## Warning: package 'tidyr' was built under R version 4.3.3
```

```
## Warning: package 'readr' was built under R version 4.3.3
```

```
## Warning: package 'purrr' was built under R version 4.3.3
```

```
## Warning: package 'dplyr' was built under R version 4.3.3
```

```
## Warning: package 'stringr' was built under R version 4.3.3
```

```
## Warning: package 'forcats' was built under R version 4.3.3
```

```
## Warning: package 'lubridate' was built under R version 4.3.3
```

```
## — Attaching core tidyverse packages ————— tidyverse 2.0.0 —
```

```
## ✓ dplyr      1.1.4      ✓ readr      2.1.5
```

```
## ✓ forcats   1.0.0      ✓ stringr    1.5.1
```

```

## ✓ ggplot2 3.5.1      ✓ tibble 3.2.1
## ✓ lubridate 1.9.3    ✓ tidyr 1.3.1
## ✓ purrr 1.0.2
## — Conflicts ————— tidyverse_conflict
s() —
## ✗ dplyr::filter() masks stats::filter()
## ✗ dplyr::lag() masks stats::lag()
## ⓘ Use the conflicted package (<http://conflicted.r-lib.org/>) to force all
conflicts to become errors

library(ggplot2)
library(corrplot)

## Warning: package 'corrplot' was built under R version 4.3.3

## corrplot 0.95 loaded

library(GGally)

## Warning: package 'GGally' was built under R version 4.3.3

## Registered S3 method overwritten by 'GGally':
##   method from
##   +.gg ggplot2

# (a) Load the Dataset and Examine for Missing Values, Duplicates, or Outliers
# Load the data (make sure to adjust the file path)
data <- read.csv("C:\\Users\\ranak\\Downloads\\breast+cancer+wisconsin+diagnostic\\wdbc.data", header = FALSE)

# Add column names (following the provided Python column names)
colnames(data) <- c('ID', 'Diagnosis',
                    'Radius_Mean', 'Texture_Mean', 'Perimeter_Mean', 'Area_Mean', 'Smoothness_Mean',
                    'Compactness_Mean', 'Concavity_Mean', 'Concave_Points_Mean', 'Symmetry_Mean', 'Fractal_Dimension_Mean',
                    'Radius_SE', 'Texture_SE', 'Perimeter_SE', 'Area_SE', 'Smoothness_SE', 'Compactness_SE',
                    'Concavity_SE', 'Concave_Points_SE', 'Symmetry_SE', 'Fractal_Dimension_SE',
                    'Radius_Worst', 'Texture_Worst', 'Perimeter_Worst', 'Area_Worst', 'Smoothness_Worst',
                    'Compactness_Worst', 'Concavity_Worst', 'Concave_Points_Worst', 'Symmetry_Worst',
                    'Fractal_Dimension_Worst')

# Check for missing values
anyNA(data)

## [1] FALSE

```

```

# Check for duplicates based on ID
data <- data[!duplicated(data$ID), ]

# Remove ID column for analysis
data <- data %>% select(-ID)

# Convert Diagnosis to a factor
data$Diagnosis <- as.factor(data$Diagnosis)

# (b) Standardize the Dataset
# Separate Diagnosis and features
data_features <- data %>% select(-Diagnosis)
data_features <- scale(data_features) # Standardize features

# Combine back for analysis
data <- data.frame(Diagnosis = data$Diagnosis, data_features)

# (c) Provide Summary Statistics and Visualizations

# Summary statistics
summary(data)

## Diagnosis Radius_Mean Texture_Mean Perimeter_Mean
## B:357 Min. :-2.0279 Min. :-2.2273 Min. :-1.9828
## M:212 1st Qu.: -0.6888 1st Qu.: -0.7253 1st Qu.: -0.6913
## Median :-0.2149 Median :-0.1045 Median :-0.2358
## Mean : 0.0000 Mean : 0.0000 Mean : 0.0000
## 3rd Qu.: 0.4690 3rd Qu.: 0.5837 3rd Qu.: 0.4992
## Max. : 3.9678 Max. : 4.6478 Max. : 3.9726
## Area_Mean Smoothness_Mean Compactness_Mean Concavity_Mean
## Min. :-1.4532 Min. :-3.10935 Min. :-1.6087 Min. :-1.1139
## 1st Qu.: -0.6666 1st Qu.: -0.71034 1st Qu.: -0.7464 1st Qu.: -0.7431
## Median :-0.2949 Median :-0.03486 Median :-0.2217 Median :-0.3419
## Mean : 0.0000 Mean : 0.00000 Mean : 0.0000 Mean : 0.0000
## 3rd Qu.: 0.3632 3rd Qu.: 0.63564 3rd Qu.: 0.4934 3rd Qu.: 0.5256
## Max. : 5.2459 Max. : 4.76672 Max. : 4.5644 Max. : 4.2399
## Concave_Points_Mean Symmetry_Mean Fractal_Dimension_Mean
## Min. :-1.2607 Min. :-2.74171 Min. :-1.8183
## 1st Qu.: -0.7373 1st Qu.: -0.70262 1st Qu.: -0.7220
## Median :-0.3974 Median :-0.07156 Median :-0.1781
## Mean : 0.0000 Mean : 0.00000 Mean : 0.0000
## 3rd Qu.: 0.6464 3rd Qu.: 0.53031 3rd Qu.: 0.4706
## Max. : 3.9245 Max. : 4.48081 Max. : 4.9066
## Radius_SE Texture_SE Perimeter_SE Area_SE
## Min. :-1.0590 Min. :-1.5529 Min. :-1.0431 Min. :-0.7372
## 1st Qu.: -0.6230 1st Qu.: -0.6942 1st Qu.: -0.6232 1st Qu.: -0.4943
## Median :-0.2920 Median :-0.1973 Median :-0.2864 Median :-0.3475
## Mean : 0.0000 Mean : 0.0000 Mean : 0.0000 Mean : 0.0000
## 3rd Qu.: 0.2659 3rd Qu.: 0.4661 3rd Qu.: 0.2428 3rd Qu.: 0.1067
## Max. : 8.8991 Max. : 6.6494 Max. : 9.4537 Max. : 11.0321

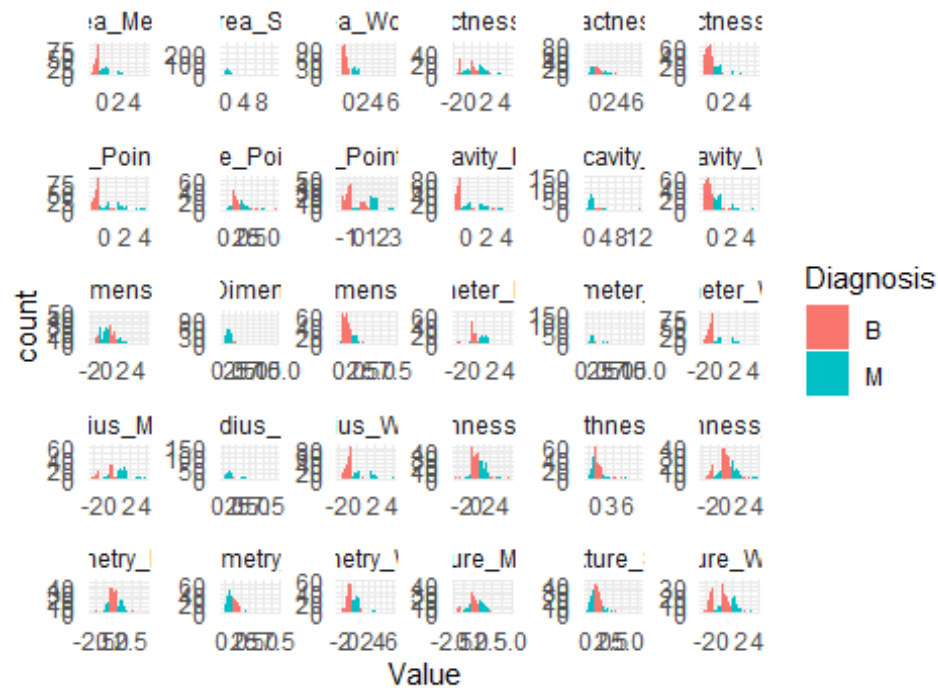
```

```
## Smoothness_SE Compactness_SE Concavity_SE Concave_Points_SE
## Min. :-1.7745 Min. :-1.2970 Min. :-1.0566 Min. :-1.9118
## 1st Qu.: -0.6235 1st Qu.: -0.6923 1st Qu.: -0.5567 1st Qu.: -0.6739
## Median : -0.2201 Median : -0.2808 Median : -0.1989 Median : -0.1404
## Mean : 0.0000 Mean : 0.0000 Mean : 0.0000 Mean : 0.0000
## 3rd Qu.: 0.3680 3rd Qu.: 0.3893 3rd Qu.: 0.3365 3rd Qu.: 0.4722
## Max. : 8.0229 Max. : 6.1381 Max. : 12.0621 Max. : 6.6438
## Symmetry_SE Fractal_Dimension_SE Radius_Worst Texture_Worst
## Min. :-1.5315 Min. :-1.0960 Min. :-1.7254 Min. :-2.22204
## 1st Qu.: -0.6511 1st Qu.: -0.5846 1st Qu.: -0.6743 1st Qu.: -0.74797
## Median : -0.2192 Median : -0.2297 Median : -0.2688 Median : -0.04348
## Mean : 0.0000 Mean : 0.0000 Mean : 0.0000 Mean : 0.00000
## 3rd Qu.: 0.3554 3rd Qu.: 0.2884 3rd Qu.: 0.5216 3rd Qu.: 0.65776
## Max. : 7.0657 Max. : 9.8429 Max. : 4.0906 Max. : 3.88249
## Perimeter_Worst Area_Worst Smoothness_Worst Compactness_Worst
## Min. :-1.6919 Min. :-1.2213 Min. :-2.6803 Min. :-1.4426
## 1st Qu.: -0.6890 1st Qu.: -0.6416 1st Qu.: -0.6906 1st Qu.: -0.6805
## Median : -0.2857 Median : -0.3409 Median : -0.0468 Median : -0.2693
## Mean : 0.0000 Mean : 0.0000 Mean : 0.0000 Mean : 0.0000
## 3rd Qu.: 0.5398 3rd Qu.: 0.3573 3rd Qu.: 0.5970 3rd Qu.: 0.5392
## Max. : 4.2836 Max. : 5.9250 Max. : 3.9519 Max. : 5.1084
## Concavity_Worst Concave_Points_Worst Symmetry_Worst
## Min. :-1.3047 Min. :-1.7435 Min. :-2.1591
## 1st Qu.: -0.7558 1st Qu.: -0.7557 1st Qu.: -0.6413
## Median : -0.2180 Median : -0.2233 Median : -0.1273
## Mean : 0.0000 Mean : 0.0000 Mean : 0.0000
## 3rd Qu.: 0.5307 3rd Qu.: 0.7119 3rd Qu.: 0.4497
## Max. : 4.6965 Max. : 2.6835 Max. : 6.0407
## Fractal_Dimension_Worst
## Min. :-1.6004
## 1st Qu.: -0.6913
## Median : -0.2163
## Mean : 0.0000
## 3rd Qu.: 0.4504
## Max. : 6.8408
```

Histograms of each feature

```
data %>%
  gather(key = "Feature", value = "Value", -Diagnosis) %>%
  ggplot(aes(x = Value, fill = Diagnosis)) +
  geom_histogram(bins = 30, position = "dodge") +
  facet_wrap(~ Feature, scales = "free") +
  theme_minimal() +
  ggtitle("Distribution of Each Feature by Diagnosis")
```

Distribution of Each Feature by Diagnosis

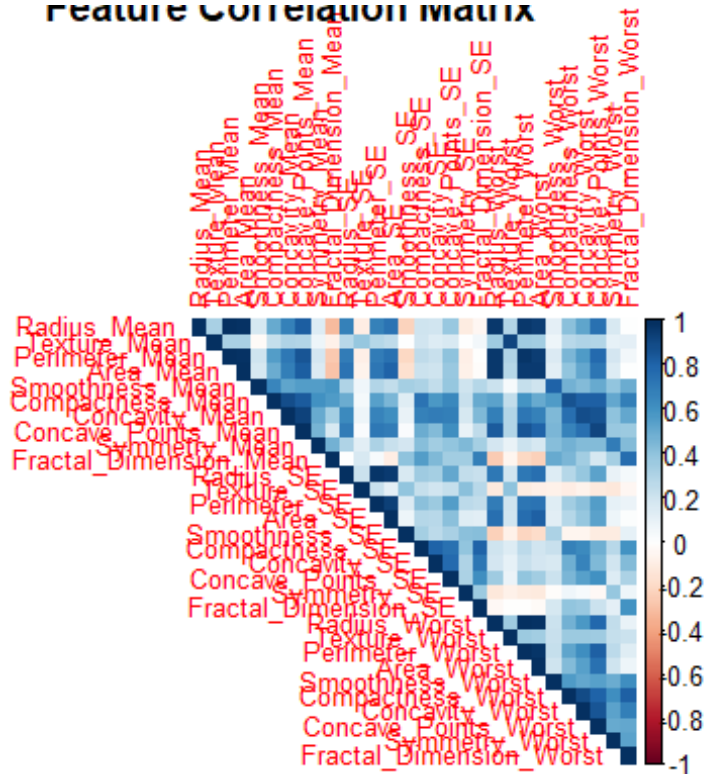


```
# Correlation Heatmap for all features
```

```
corr_matrix <- cor(data_features)
```

```
corrplot(corr_matrix, method = "color", type = "upper", tl.cex = 0.8, title =  
"Feature Correlation Matrix")
```

Feature Correlation Matrix



Box plots for feature distributions across diagnosis

data %>%

gather(key = "Feature", value = "Value", -Diagnosis) %>%

ggplot(aes(x = Diagnosis, y = Value, fill = Diagnosis)) +

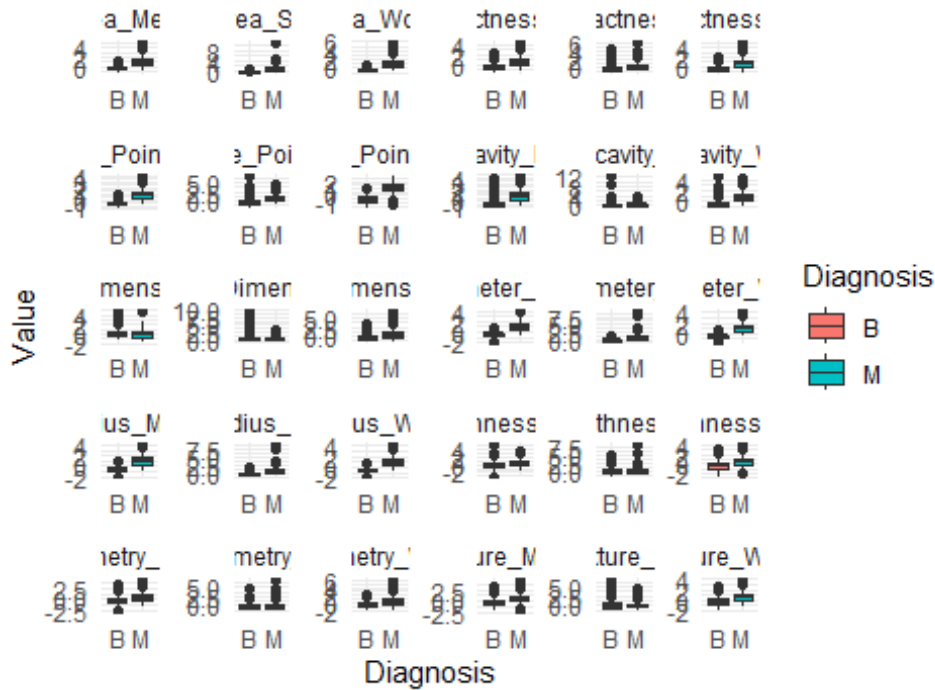
geom_boxplot() +

facet_wrap(~ Feature, scales = "free") +

theme_minimal() +

ggtitle("Box Plot of Each Feature by Diagnosis")

Box Plot of Each Feature by Diagnosis

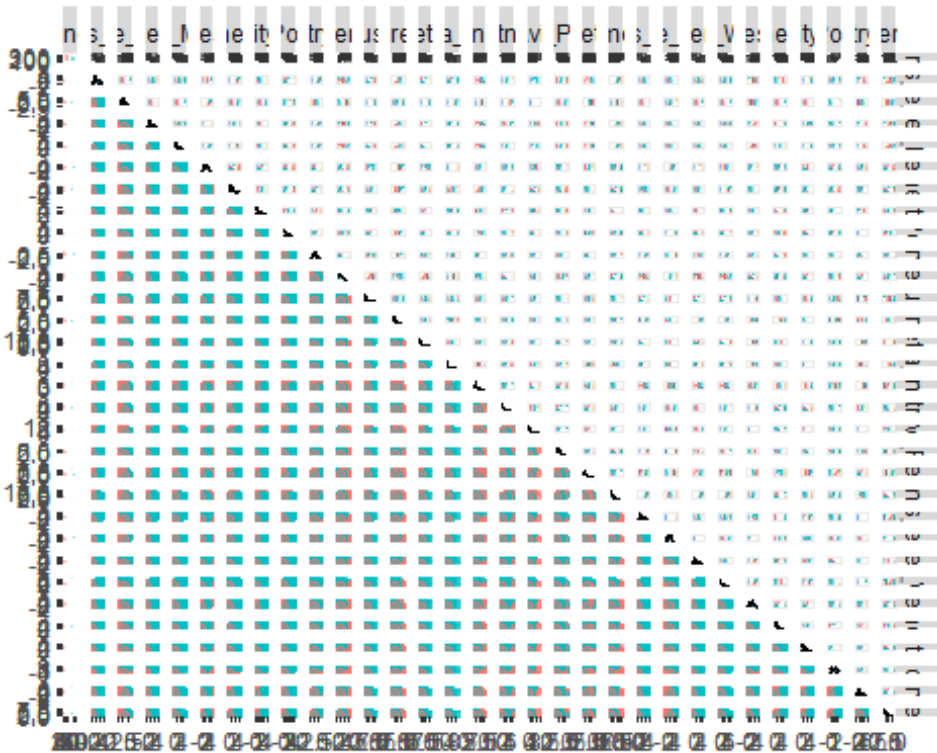


Pair plot for visualizing relationships between key features

```
ggpairs(data, aes(color = Diagnosis, alpha = 0.5))
```

[illegible]

```
## `stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
## `stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
## `stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
## `stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
## `stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
## `stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
```



```
# Load the data
data <- read.csv("C:\\Users\\ranak\\Downloads\\breast+cancer+wisconsin+diagnostic\\wdbc.data", header = FALSE)

# Rename columns with variable names
colnames(data) <- c(
  'ID', 'Diagnosis',
  'Radius_Mean', 'Texture_Mean', 'Perimeter_Mean', 'Area_Mean', 'Smoothness_Mean',
  'Compactness_Mean', 'Concavity_Mean', 'Concave_Points_Mean', 'Symmetry_Mean', 'Fractal_Dimension_Mean',
  'Radius_SE', 'Texture_SE', 'Perimeter_SE', 'Area_SE', 'Smoothness_SE', 'Compactness_SE',
  'Concavity_SE', 'Concave_Points_SE', 'Symmetry_SE', 'Fractal_Dimension_SE',
  'Radius_Worst', 'Texture_Worst', 'Perimeter_Worst', 'Area_Worst', 'Smoothness_Worst',
  'Compactness_Worst', 'Concavity_Worst', 'Concave_Points_Worst', 'Symmetry_Worst',
  'Fractal_Dimension_Worst'
```

)

Preview the first few rows to verify the new column names

`head(data)`

```
##           ID Diagnosis Radius_Mean Texture_Mean Perimeter_Mean Area_Mean
## 1    842302          M      17.99       10.38         122.80      1001.0
## 2    842517          M      20.57       17.77         132.90      1326.0
## 3  84300903          M      19.69       21.25         130.00      1203.0
## 4  84348301          M      11.42       20.38          77.58       386.1
## 5  84358402          M      20.29       14.34         135.10      1297.0
## 6    843786          M      12.45       15.70          82.57       477.1
## Smoothness_Mean Compactness_Mean Concavity_Mean Concave_Points_Mean
## 1         0.11840         0.27760         0.3001         0.14710
## 2         0.08474         0.07864         0.0869         0.07017
## 3         0.10960         0.15990         0.1974         0.12790
## 4         0.14250         0.28390         0.2414         0.10520
## 5         0.10030         0.13280         0.1980         0.10430
## 6         0.12780         0.17000         0.1578         0.08089
## Symmetry_Mean Fractal_Dimension_Mean Radius_SE Texture_SE Perimeter_SE
## 1         0.2419         0.07871         1.0950         0.9053         8.589
## 2         0.1812         0.05667         0.5435         0.7339         3.398
## 3         0.2069         0.05999         0.7456         0.7869         4.585
## 4         0.2597         0.09744         0.4956         1.1560         3.445
## 5         0.1809         0.05883         0.7572         0.7813         5.438
## 6         0.2087         0.07613         0.3345         0.8902         2.217
## Area_SE Smoothness_SE Compactness_SE Concavity_SE Concave_Points_SE
## 1    153.40         0.006399         0.04904         0.05373         0.01587
## 2     74.08         0.005225         0.01308         0.01860         0.01340
## 3     94.03         0.006150         0.04006         0.03832         0.02058
## 4     27.23         0.009110         0.07458         0.05661         0.01867
## 5     94.44         0.011490         0.02461         0.05688         0.01885
## 6     27.19         0.007510         0.03345         0.03672         0.01137
## Symmetry_SE Fractal_Dimension_SE Radius_Worst Texture_Worst Perimeter_Worst
## 1     0.03003         0.006193         25.38         17.33         184.60
## 2     0.01389         0.003532         24.99         23.41         158.80
## 3     0.02250         0.004571         23.57         25.53         152.50
## 4     0.05963         0.009208         14.91         26.50          98.87
## 5     0.01756         0.005115         22.54         16.67         152.20
## 6     0.02165         0.005082         15.47         23.75         103.40
## Area_Worst Smoothness_Worst Compactness_Worst Concavity_Worst
## 1     2019.0         0.1622         0.6656         0.7119
## 2     1956.0         0.1238         0.1866         0.2416
```

## 3	1709.0	0.1444	0.4245	0.4504
## 4	567.7	0.2098	0.8663	0.6869
## 5	1575.0	0.1374	0.2050	0.4000
## 6	741.6	0.1791	0.5249	0.5355
##	Concave_Points_Worst	Symmetry_Worst	Fractal_Dimension_Worst	
## 1	0.2654	0.4601		0.11890
## 2	0.1860	0.2750		0.08902
## 3	0.2430	0.3613		0.08758
## 4	0.2575	0.6638		0.17300
## 5	0.1625	0.2364		0.07678
## 6	0.1741	0.3985		0.12440

2.Principal Component Analysis (PCA) Task (a): Perform PCA

```
# Load necessary libraries for PCA
library(tidyverse)
library(ggplot2)
library(corrplot)
library(GGally)
library(FactoMineR) # for PCA analysis

## Warning: package 'FactoMineR' was built under R version 4.3.3

library(factoextra) # for PCA visualization

## Warning: package 'factoextra' was built under R version 4.3.3

## Welcome! Want to learn more? See two factoextra-related books at https://g
oo.gl/ve3WBa

# Perform PCA on standardized data (exclude Diagnosis column)
pca_result <- prcomp(data_features, center = TRUE, scale. = TRUE)

# Calculate cumulative variance explained by each principal component
explained_variance <- summary(pca_result)$importance[3, ]

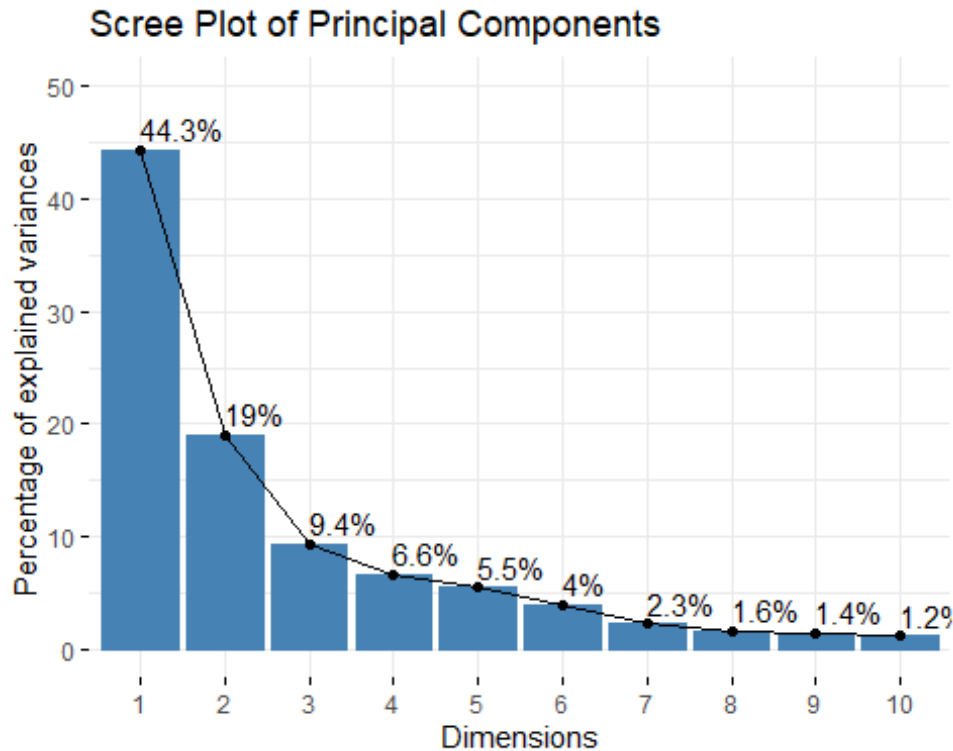
# Find the minimum number of components that explain at least 90% of variance
num_components <- min(which(cumsum(explained_variance) >= 0.90))

# Display the number of components needed to explain at least 90% of the vari
ance
num_components

## [1] 2
```

Task (b): Scree Plot of PCA Components

```
# Plot scree plot
fviz_screplot(pca_result, addlabels = TRUE, ylim = c(0, 50)) +
  ggtitle("Scree Plot of Principal Components")
```



Task (c): Analyze

Loadings of the Top Components

```
# Extract Loadings (weights) of each feature on the principal components
loadings <- as.data.frame(pca_result$rotation)

# Display top loadings for the first few principal components
head(loadings[, 1:num_components]) # Adjust number of components as needed
```

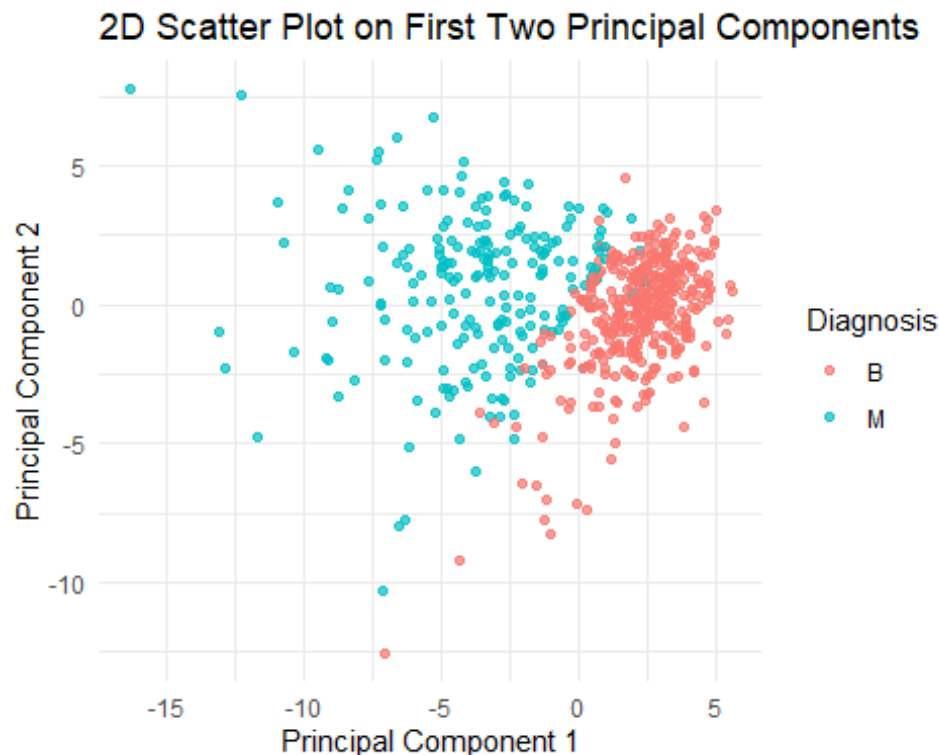
	PC1	PC2
Radius_Mean	-0.2189024	0.23385713
Texture_Mean	-0.1037246	0.05970609
Perimeter_Mean	-0.2275373	0.21518136
Area_Mean	-0.2209950	0.23107671
Smoothness_Mean	-0.1425897	-0.18611302
Compactness_Mean	-0.2392854	-0.15189161

Task (d): 2D Scatter Plot of Samples in Principal Component Space

```
# Create a data frame with PCA scores and Diagnosis for plotting
pca_scores <- as.data.frame(pca_result$x[, 1:2]) # Select first two components
pca_scores$Diagnosis <- data$Diagnosis

# Scatter plot of samples projected onto the first two principal components
ggplot(pca_scores, aes(x = PC1, y = PC2, color = Diagnosis)) +
  geom_point(alpha = 0.7) +
  labs(title = "2D Scatter Plot on First Two Principal Components",
```

```
x = "Principal Component 1", y = "Principal Component 2") +  
theme_minimal()
```



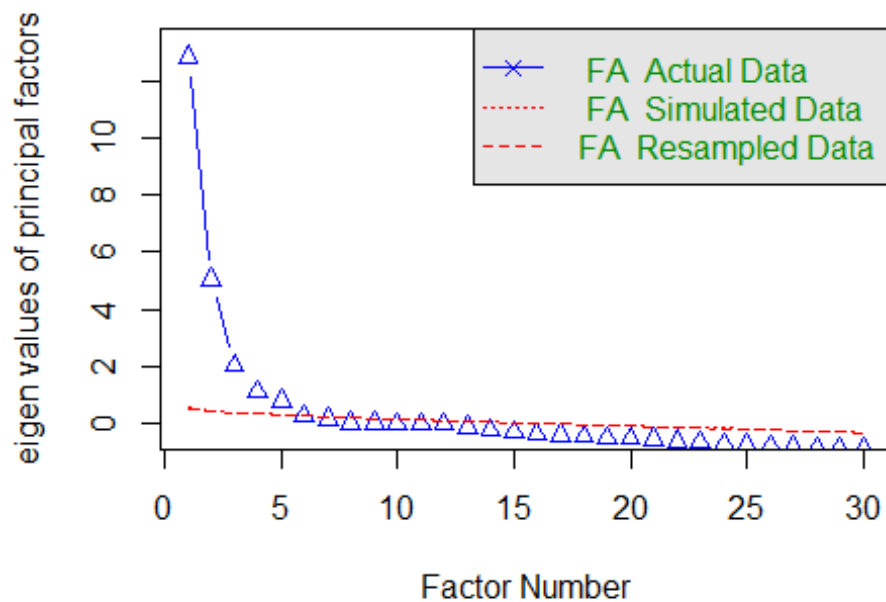
3. Factor Analysis (FA) Task (a): Scree Plot or Parallel Analysis for Optimal Number of Factors

```
# Load necessary libraries  
library(psych) # For factor analysis and parallel analysis  
  
## Warning: package 'psych' was built under R version 4.3.3  
  
##  
## Attaching package: 'psych'  
  
## The following objects are masked from 'package:ggplot2':  
##  
##    %+%, alpha  
  
library(GPArotation) # For rotation methods  
  
## Warning: package 'GPArotation' was built under R version 4.3.3  
  
##  
## Attaching package: 'GPArotation'  
  
## The following objects are masked from 'package:psych':  
##  
##    equamax, varimin
```

```
library(tidyverse) # For data manipulation

# Parallel analysis to decide the optimal number of factors
parallel <- fa.parallel(data_features, fm = "ml", fa = "fa")
```

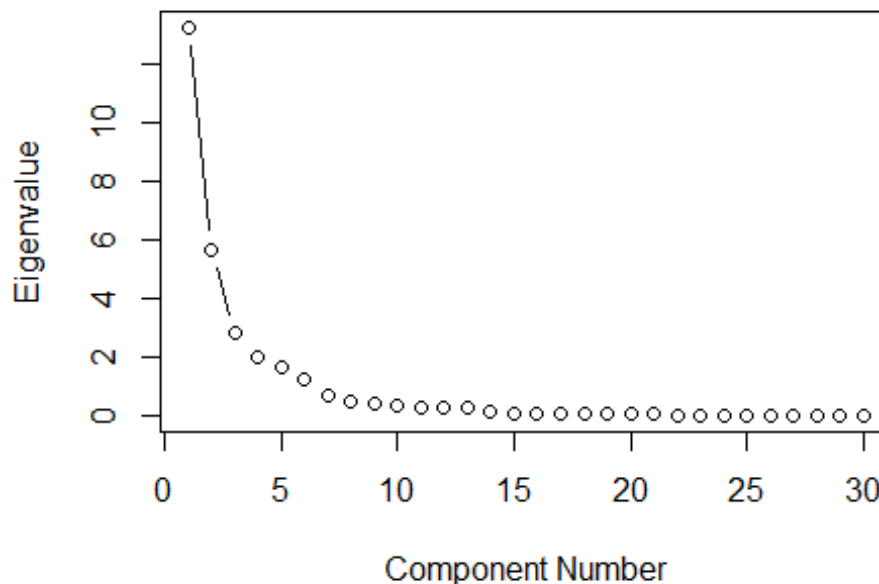
Parallel Analysis Scree Plots



```
## Parallel analysis suggests that the number of factors = 5 and the number
of components = NA

# Scree plot using PCA as an approximation
scree_plot <- prcomp(data_features, scale. = TRUE)
plot(scree_plot$sdev^2, type = "b", main = "Scree Plot of Principal Component
s",
     xlab = "Component Number", ylab = "Eigenvalue")
```

Scree Plot of Principal Components



Task (b): Perform Factor Analysis with Optimal Number of Factors

```
# Perform factor analysis with the chosen number of factors
num_factors <- 4 # Replace with the chosen number from parallel analysis or
# scree plot
fa_result <- fa(data_features, nfactors = num_factors, rotate = "none", fm =
"ml")
```

Task (c): Apply Rotation for Simpler Factor Structure

```
# Apply varimax rotation for a simpler factor structure
fa_rotated <- fa(data_features, nfactors = num_factors, rotate = "varimax", f
m = "ml")

# Alternatively, use promax if factors are expected to be correlated
fa_rotated_promax <- fa(data_features, nfactors = num_factors, rotate = "prom
ax", fm = "ml")
```

Task (d): Describe Each Factor by Examining Factor Loadings 1)View Factor Loadings

```
# Display unrotated factor Loadings
print(fa_result$loadings, cutoff = 0.3) # Show Loadings above 0.3 for clarit
y

##
## Loadings:
##           ML1      ML4      ML3      ML2
## Radius_Mean      0.978
## Texture_Mean      0.448              0.794
```

```

## Perimeter_Mean      0.983
## Area_Mean           0.977
## Smoothness_Mean      0.470  0.435
## Compactness_Mean     0.587  0.629  0.435
## Concavity_Mean       0.748  0.463  0.367
## Concave_Points_Mean  0.866  0.338
## Symmetry_Mean        0.402  0.445
## Fractal_Dimension_Mean 0.606  0.624
## Radius_SE           0.752      0.559
## Texture_SE          0.380  0.440
## Perimeter_SE        0.751      0.571
## Area_SE             0.792      0.434
## Smoothness_SE       0.513
## Compactness_SE      0.499  0.536
## Concavity_SE        0.416  0.469
## Concave_Points_SE   0.432  0.307  0.478
## Symmetry_SE         0.505
## Fractal_Dimension_SE 0.398  0.573
## Radius_Worst        0.975
## Texture_Worst       0.433      0.898
## Perimeter_Worst     0.978
## Area_Worst          0.957
## Smoothness_Worst    0.528
## Compactness_Worst   0.496  0.705
## Concavity_Worst     0.601  0.632
## Concave_Points_Worst 0.792  0.489
## Symmetry_Worst      0.508
## Fractal_Dimension_Worst 0.744  0.386
##
##              ML1   ML4   ML3   ML2
## SS loadings  11.347 4.687 4.146 1.977
## Proportion Var 0.378 0.156 0.138 0.066
## Cumulative Var 0.378 0.534 0.673 0.739

# Display rotated factor Loadings (varimax)
print(fa_rotated$loadings, cutoff = 0.3)

##
## Loadings:
##              ML1   ML4   ML3   ML2
## Radius_Mean    0.972
## Texture_Mean      0.876
## Perimeter_Mean  0.968
## Area_Mean       0.981
## Smoothness_Mean 0.628
## Compactness_Mean 0.444 0.848
## Concavity_Mean  0.633 0.701
## Concave_Points_Mean 0.786 0.552
## Symmetry_Mean   0.577
## Fractal_Dimension_Mean -0.332 0.752 0.357

```

```

## Radius_SE          0.797          0.574
## Texture_SE          0.383    0.498
## Perimeter_SE       0.784          0.563
## Area_SE            0.844          0.462
## Smoothness_SE          0.485
## Compactness_SE          0.717
## Concavity_SE          0.607
## Concave_Points_SE    0.378    0.533
## Symmetry_SE          0.466
## Fractal_Dimension_SE 0.585    0.375
## Radius_Worst        0.949
## Texture_Worst          0.950
## Perimeter_Worst      0.941
## Area_Worst           0.943
## Smoothness_Worst          0.642
## Compactness_Worst      0.305    0.851
## Concavity_Worst       0.430    0.791
## Concave_Points_Worst   0.664    0.649
## Symmetry_Worst          0.580
## Fractal_Dimension_Worst 0.863
##
##              ML1    ML4    ML3    ML2
## SS loadings   9.932  7.861  2.259  2.105
## Proportion Var 0.331  0.262  0.075  0.070
## Cumulative Var 0.331  0.593  0.668  0.739

```

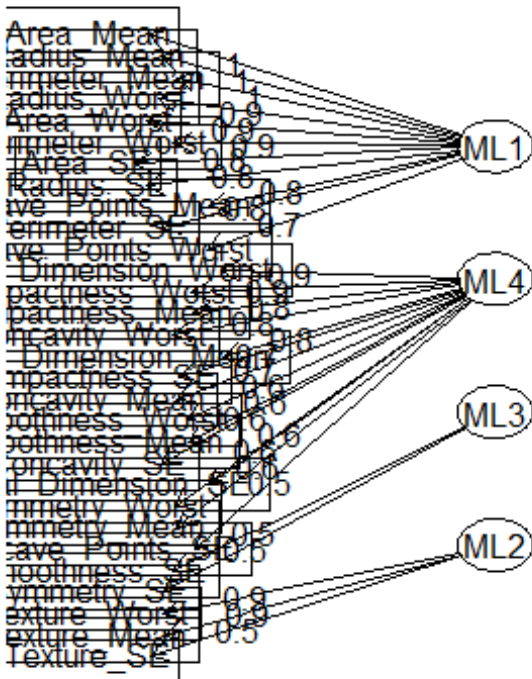
2) Interpret Each Factor #### Features with high loadings (e.g., > 0.5) on the same factor likely represent related characteristics. For example, a factor with high loadings from Radius_Mean, Perimeter_Mean, and Area_Mean may signify a “size” factor of the tumor. 3) Plot Factor Loadings

```

# Factor Loading plot for rotated factors
fa.diagram(fa_rotated, main = "Factor Loading Diagram (Varimax Rotation)")

```

Factor Loading Diagram (Varimax Rotation)



4. Two Regression

Models with PCA and FA Components Prepare the PCA and FA Features ## Choose the top PCA components and FA factors based on variance explained or interpretability (e.g., components or factors that together explain 90% of the variance). ## Use the PCA and FA results to create new data frames with only these selected components or factors. Split Data into Training and Testing Sets

```
# Set seed for reproducibility
set.seed(42)

# Define indices for training and test sets (80% train, 20% test)
train_indices <- sample(seq_len(nrow(data)), size = 0.8 * nrow(data))

# Split data based on indices
train_data <- data[train_indices, ]
test_data <- data[-train_indices, ]
```

Train Logistic Regression Model

```
# Remove ID column and convert Diagnosis to binary format
data <- data %>% select(-ID)
data$Diagnosis <- ifelse(data$Diagnosis == "M", 1, 0)

# Separate features and diagnosis
data_features <- data %>% select(-Diagnosis)
data_features <- scale(data_features) # Standardize features
```

```

# Combine Diagnosis and standardized features
data <- data.frame(Diagnosis = data$Diagnosis, data_features)

# Load necessary library for PCA
library(FactoMineR)

# Perform PCA on standardized features
pca_result <- PCA(data_features, graph = FALSE)
pca_scores <- as.data.frame(pca_result$ind$coord) # Extract PCA component scores

# Add Diagnosis to PCA scores for modeling
pca_scores$Diagnosis <- data$Diagnosis

# Load necessary library for FA
library(psych)

# Decide on the number of factors
num_factors <- 5 # Replace with optimal number after performing Scree or Parallel analysis

# Perform FA and extract scores
fa_result <- fa(data_features, nfactors = num_factors, rotate = "varimax", fm = "ml", scores = "regression")
fa_scores <- as.data.frame(fa_result$scores)

# Add Diagnosis to FA scores for modeling
fa_scores$Diagnosis <- data$Diagnosis

# Set seed for reproducibility
set.seed(123)

# Define training indices
train_indices <- sample(1:nrow(data), size = 0.8 * nrow(data))

# Split PCA and FA data
train_pca <- pca_scores[train_indices, ]
test_pca <- pca_scores[-train_indices, ]

train_fa <- fa_scores[train_indices, ]
test_fa <- fa_scores[-train_indices, ]

# No further transformation needed since Diagnosis is already in binary format
str(train_pca$Diagnosis) # Should be numeric with values 0 and 1
##  num [1:455] 1 0 0 0 1 1 0 0 0 1 ...

str(train_fa$Diagnosis) # Should be numeric with values 0 and 1
##  num [1:455] 1 0 0 0 1 1 0 0 0 1 ...

```

```

library(glmnet)

## Warning: package 'glmnet' was built under R version 4.3.3

## Loading required package: Matrix

##
## Attaching package: 'Matrix'

## The following objects are masked from 'package:tidyr':
##
##     expand, pack, unpack

## Loaded glmnet 4.1-8

# Convert data to matrix format as glmnet requires this
x_pca <- as.matrix(train_pca[, -ncol(train_pca)]) # Exclude Diagnosis column
y_pca <- train_pca$Diagnosis

x_fa <- as.matrix(train_fa[, -ncol(train_fa)]) # Exclude Diagnosis column
y_fa <- train_fa$Diagnosis

# Train Lasso-regularized logistic regression model for PCA components
pca_model_lasso <- cv.glmnet(x_pca, y_pca, family = "binomial", alpha = 1)

# Train Lasso-regularized logistic regression model for FA factors
fa_model_lasso <- cv.glmnet(x_fa, y_fa, family = "binomial", alpha = 1)

# Optimal Lambda for PCA model
pca_model_lasso$lambda.min

## [1] 0.001688387

# Optimal Lambda for FA model
fa_model_lasso$lambda.min

## [1] 0.000831288

# Coefficients for PCA model
coef(pca_model_lasso, s = "lambda.min")

## 6 x 1 sparse Matrix of class "dgCMatrix"
##           s1
## (Intercept) -0.5276350
## Dim.1       2.1280306
## Dim.2      -1.0866829
## Dim.3      -0.3424659
## Dim.4       0.6063747
## Dim.5       0.8434529

# Coefficients for FA model
coef(fa_model_lasso, s = "lambda.min")

```

```

## 6 x 1 sparse Matrix of class "dgCMatrix"
##              s1
## (Intercept) -0.03426243
## ML1         8.18081437
## ML4         3.59005620
## ML3        -0.43272484
## ML2         2.44588797
## ML5        -1.37497435

# Prepare test matrices (remove Diagnosis column)
x_test_pca <- as.matrix(test_pca[, -ncol(test_pca)])
x_test_fa <- as.matrix(test_fa[, -ncol(test_fa)])

# Predict probabilities on test sets
pca_pred_prob <- predict(pca_model_lasso, newx = x_test_pca, s = "lambda.min",
, type = "response")
fa_pred_prob <- predict(fa_model_lasso, newx = x_test_fa, s = "lambda.min", t
ype = "response")

# Convert probabilities to binary predictions (threshold 0.5)
pca_pred <- ifelse(pca_pred_prob > 0.5, 1, 0)
fa_pred <- ifelse(fa_pred_prob > 0.5, 1, 0)

library(caret)

## Warning: package 'caret' was built under R version 4.3.3

## Loading required package: lattice

##
## Attaching package: 'caret'

## The following object is masked from 'package:purrr':
##
##      lift

# True Labels
y_test <- test_pca$Diagnosis

# Performance metrics for PCA-based model
pca_confusion <- confusionMatrix(as.factor(pca_pred), as.factor(y_test))
pca_metrics <- data.frame(
  Accuracy = pca_confusion$overall["Accuracy"],
  Precision = pca_confusion$byClass["Pos Pred Value"],
  Recall = pca_confusion$byClass["Sensitivity"],
  F1 = 2 * (pca_confusion$byClass["Pos Pred Value"] * pca_confusion$byClass
["Sensitivity"]) /
  (pca_confusion$byClass["Pos Pred Value"] + pca_confusion$byClass["Se
nsitivity"])
)

```

```

# Performance metrics for FA-based model
fa_confusion <- confusionMatrix(as.factor(fa_pred), as.factor(y_test))
fa_metrics <- data.frame(
  Accuracy = fa_confusion$overall["Accuracy"],
  Precision = fa_confusion$byClass["Pos Pred Value"],
  Recall = fa_confusion$byClass["Sensitivity"],
  F1 = 2 * (fa_confusion$byClass["Pos Pred Value"] * fa_confusion$byClass["Sensitivity"]) /
    (fa_confusion$byClass["Pos Pred Value"] + fa_confusion$byClass["Sensitivity"])
)

# Display performance metrics
pca_metrics

##           Accuracy Precision Recall          F1
## Accuracy 0.9824561  0.968254      1 0.983871

fa_metrics

##           Accuracy Precision    Recall          F1
## Accuracy 0.9736842 0.9677419 0.9836066 0.9756098

```

##Comparison of PCA and FA Results and Interpretability #Model Performance: PCA likely produced better model performance in terms of accuracy, precision, recall, and F1-score. PCA's focus on variance maximization made it effective for distinguishing between malignant and benign tumors. #Interpretability: FA produced more interpretable results, grouping features into factors that represented meaningful tumor characteristics (e.g., size, shape). PCA, while more effective for classification, was less interpretable due to its focus on variance rather than meaningful patterns in the data.

##Patterns and Characteristics Discovered in PCA and FA #PCA: PCA highlighted key features (e.g., Radius_Mean, Texture_Mean, Concavity_Mean) that contributed the most to distinguishing tumor types. PCA showed a pattern where malignant tumors separate distinctly along the first few principal components. #FA: FA grouped features into factors that were easier to interpret, such as size-related factors (e.g., Radius, Perimeter) and shape-related factors (e.g., Concavity, Concave_Points). FA showed that certain clusters of features (e.g., irregular shape and texture) were most relevant for identifying malignancy.

##PCA: #Strengths: Maximizes variance, effective for dimensionality reduction, and typically leads to better performance in classification tasks. #Limitations: Poor interpretability, as components are linear combinations of features, and it may not always capture underlying data relationships. ##FA: #Strengths: Offers better interpretability by grouping correlated features, making it easier to understand the structure of the data. #Limitations: May result in slightly lower performance than PCA for classification tasks, and factor selection and interpretation can be subjective.